

Kubernetesの概要と運用の基本

コンテナ化されたアプリケーションを支えるオーケストレーション基盤

1. Kubernetesとは

Kubernetesは、コンテナ化されたアプリケーションの配置、スケーリング、更新、障害復旧などを自動化するためのオープンソースのコンテナオーケストレーション基盤である。個々のコンテナを直接管理するのではなく、アプリケーションがどのような状態で動作すべきかを宣言し、その状態を継続的に維持する仕組みを提供する。これにより、サーバー台数や配置先を人手で細かく管理する必要を減らし、複数ノードにまたがるアプリケーションを一貫した方法で運用できる。

Kubernetesでは「望ましい状態」をAPIオブジェクトとして登録する。たとえば、Webアプリケーションを3個のPodで常時稼働させると宣言すると、Kubernetesは実際の稼働状態を監視し、Podが停止した場合には新しいPodを起動して差分を埋める。このReconciliationと呼ばれる考え方が、Kubernetesの自動化を理解する上で重要な基礎となる。

2. クラスタを構成する主要コンポーネント

Kubernetesクラスタは、大きくControl PlaneとWorker Nodeに分かれる。Control Planeはクラスタ全体の状態を管理し、Worker Nodeは実際のアプリケーションを実行する。主要なコンポーネントは次のとおりである。

- kube-apiserver: Kubernetes APIを提供する中心的なコンポーネント。kubectlや各種Controllerからの要求を受け付ける。
- etcd: クラスタの設定や状態を保存する分散Key-Value Store。Control Planeの重要な永続データを保持する。
- kube-scheduler: 新しく作成されたPodを、リソース量や制約条件を考慮して適切なNodeへ割り当てる。
- kube-controller-manager: DeploymentやNodeなどの状態を監視し、望ましい状態との差分を解消する複数のControllerを実行する。
- kubelet: 各Worker Node上で動作し、割り当てられたPodが指定どおり実行されるようにコンテナランタイムを制御する。

3. Pod、Deployment、Service

Kubernetesで最小の実行単位となるのがPodである。Podには1つ以上のコンテナを含めることができ、同一Pod内のコンテナはネットワーク名前空間などを共有する。ただし、実運用ではPodを直接作成するよりもDeploymentなどのWorkload APIを利用することが一般的である。DeploymentはPodの複製数や更新方法を管理し、ローリングアップデートやロールバックを実現する。

一方、Podは再作成によってIPアドレスが変わるため、クライアントが個々のPodを直接参照する設計は安定しない。Serviceは複数のPodを論理的にまとめ、固定の仮想IPやDNS名を提供することで、アプリケーション間の通信先を安定化させる。

4. 宣言的管理とYAMLマニフェスト

Kubernetesでは、多くのリソースをYAML形式のマニフェストで定義する。マニフェストにはapiVersion、kind、metadata、specなどを記述し、kubectl applyなどを利用してクラスタへ反映する。手作業でサーバーへログインして設定を変更する方式と比べ、設定をコードとして保存しやすく、Gitによる履歴管理やレビュー、CI/CDとの連携が容易になる。

また、宣言的管理では操作手順そのものではなく、最終的に実現したい状態を記述する。この性質により、同じマニフェストを複数環境へ適用したり、GitOpsツールを使ってリポジトリの内容とクラスタ状態を継続的に同期したりできる。

5. スケーリングと可用性

Kubernetesはアプリケーションの可用性を高めるための仕組みを複数持つ。Deploymentのreplicasを増やせばPodを水平に複製でき、HorizontalPodAutoscalerを利用すればCPU使用率などのメトリクスに応じてPod数を自動調整できる。Node障害が発生した場合も、Control Planeが異常を検知し、必要に応じて別Node上でPodを再作成する。

ただし、Kubernetesを導入するだけで高可用性が自動的に完成するわけではない。Podを複数Nodeへ分散させるためのTopology Spread ConstraintsやPod Anti-Affinity、適切なreadinessProbe、PodDisruptionBudgetなどを組み合わせ、障害時やメンテナンス時の挙動を設計する必要がある。

6. 運用時に意識すべきポイント

本番環境では、アプリケーションの配置だけでなく、監視、ログ、セキュリティ、バックアップ、アップグレードまで含めた運用設計が必要となる。特にKubernetesは多数のAPIオブジェクトとControllerが連携するため、障害発生時には「どのリソースが期待状態から外れているか」を追跡できる観測性が重要である。

- 監視: Node、Pod、コンテナ、Control Planeのメトリクスを収集し、リソース不足や異常を検知する。
- ログ: アプリケーションログをNode外へ集約し、Podの再作成後も調査できる状態にする。
- セキュリティ: RBAC、NetworkPolicy、Secret管理、Pod Security Standardsなどを用いて権限と通信範囲を制御する。
- アップグレード: Kubernetes本体だけでなく、CNI、CSI、Ingress/Gateway、監視系コンポーネントの互換性も確認する。

7. まとめ

Kubernetesの本質は、コンテナを起動する機能そのものではなく、宣言された望ましい状態をクラスタ全体で継続的に維持する制御システムにある。Pod、Deployment、Serviceなどの基本リソースとControl Planeの役割を理解すると、スケーリングや自己修復、ローリングアップデートといった機能が同じ設計思想の上に成り立っていることが分かる。実運用では、可用性、監視、セキュリティ、更新手順まで含めて設計することで、Kubernetesの自動化能力を安定したプラットフォーム運用につなげられる。